

리팩토링 자세히보기 Step

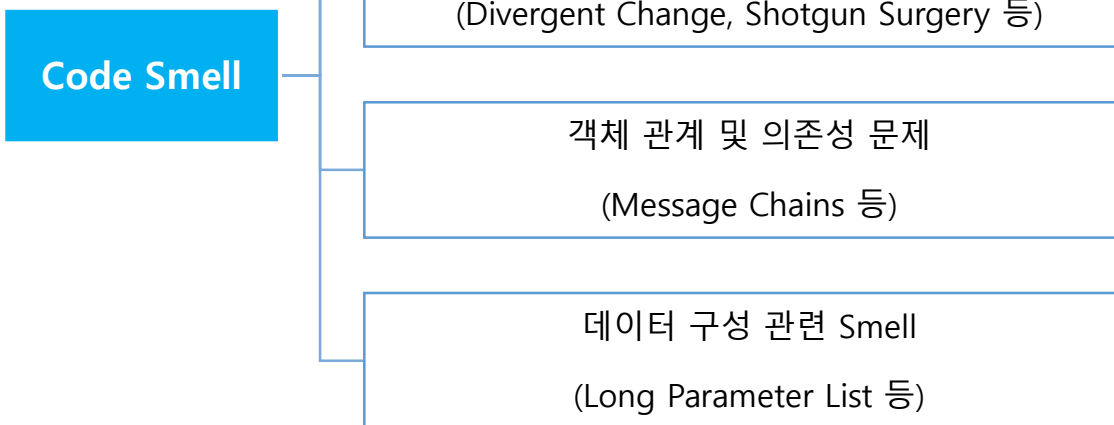
[STEP 1] 리팩토링 기법에 자주 사용되는 기본기를 살펴본다.

[STEP 2] 코드에서 유지보수성 / 확장성에 문제가 될 수 있는 잠재적 결함 (code smell)에 대해 알아보며,
각 증상을 해결하는 리팩토링 기법에 대해 자세히 다룬다.

[STEP 1]



[STEP 2]



[STEP 1] 리팩토링 기본기

- 리팩토링에서 자주 활용되는 기초 작업



리팩토링시
자주 사용되는 기법

적절한 단위로, 테스트가 항상 가능하도록!

리팩토링 기법을 할 때는 테스트 코드를 갖춘 상태에서 리팩토링을 해야 한다.

새로 만들 요소가 있다면 만든 뒤, 정적 테스트를 수행한다.

기존 요소는 적당한 단위의 리팩토링을 마친 뒤, 단위 테스트를 수행한다.

리팩토링 진행 중, 코드가 깨져선 안된다.

[리팩토링 기법] 변수 쪼개기 (Split Temporary Variable)

하나의 변수가 다양한 목적으로 재사용 되는 경우, 코드 읽는 이에게 혼란을 준다.

대입이 2번 이상 이뤄진다면 변수가 여러 목적으로 사용되고 있을 수 있다.

```
double temp = 2 * ( _height + _width );  
System.out.println(temp);  
//....  
temp = _height * _width;  
System.out.println(temp);
```



```
final double hap = 2 * ( _height + _width );  
System.out.println(hap);  
//....  
final double area = _height * _width;  
System.out.println(area);
```

final 을 이용해서 재할당을 방지할 수 있다

[참고] final 과 불변성

java 에서 final 변수는 재할당을 할 수 없다. 즉, 참조가 고정된다.

```
class Something {
    public final int a;
    public int b;
    public Something() { a = 3;}
    public void plus(){ b ++;}
}

public class Main {
    public static void main(String[] args) {
        final int a = 3;
        a ++;

        final Something s = new Something();
        s = new Something();

        s.a = 3;
        s.b = 7;
        s.plus();
    }
}
```

[리팩토링 기법] 문장 이동 (Move Statements)

덜 중요한 변수일수록 변수의 범위(스코프)를 작게 가지면 좋다.

특히, 함수 추출이 쉽게 되려면 같이 추출될 변수와 동작들이 같이 모여 있어야 한다.

따라서 변수가 변수를 사용하는 곳과 떨어져 있으면 이를 모아준다.

```
int result = 0 ;
```

```
// ...  
// ...
```

```
for(int i = 0 ; i < 1000; i ++){  
    result += doSomething();  
}
```



사용되는 곳과 코드간 거리가 멀다

```
// ...  
// ...
```

```
int result = 0 ;  
for(int i = 0 ; i < 1000; i ++){  
    result += doSomething();  
}
```

함수 추출도 쉽고 변수 파악도 쉽다

[리팩토링 기법] 함수 추출하기 (Extract Function)

코드의 의도가 잘 드러나도록 서브 함수들로 추출한다.

```
class InvoicePrinter {
    public void printInvoice(Invoice invoice){
        double totalAmount = 0;

        // Print header
        System.out.println("***** Invoice *****");
        System.out.println("날짜: " + invoice.getDate());
        System.out.println("고객: " + invoice.getCustomer().getName());

        // Print line items
        for (Item item : invoice.getItems()) {
            double amount = item.getPrice()
                * item.getQuantity();
            System.out.println(item.getName()
                + ": " + amount);
            totalAmount += amount;
        }

        // Print footer
        System.out.println("총액: " + totalAmount);
    }
}
```

```
class InvoicePrinter {
    public void printInvoice(Invoice invoice) {
        printHeader(invoice);
        double totalAmount = printLineItems(invoice);
        printFooter(totalAmount);
    }

    private void printHeader(Invoice invoice) {
        System.out.println("***** Invoice *****");
        System.out.println("날짜: " + invoice.getDate());
        System.out.println("고객: " + invoice.getCustomer().getName());
    }

    private double printLineItems(Invoice invoice) {
        double totalAmount = 0;
        for (Item item : invoice.getItems()) {
            double amount = item.getPrice() * item.getQuantity();
            System.out.println(item.getName() + ": " + amount);
            totalAmount += amount;
        }
        return totalAmount;
    }

    private void printFooter(double totalAmount) {
        System.out.println("총액: " + totalAmount);
    }
}
```

[리팩토링 기법] loop 쪼개기 (Split Loop)

loop 에 여러 기능이 섞여 있는 경우, 각 기능별 loop 로 분리할 수 있다.
분리된 loop 들은 각각 한가지 기능을 하는 함수들로 추출이 가능해진다.

```
ArrayList<Integer> numbers =  
    new ArrayList<>(List.of(1, 2, 3, 4, 5));  
  
int sum = 0;  
for (int i = 0; i < numbers.size(); i++) {  
    numbers.set(i, numbers.get(i) * 2);  
    sum += numbers.get(i);  
}  
  
System.out.println(sum);
```



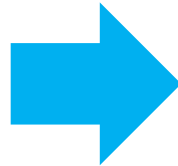
```
ArrayList<Integer> numbers =  
    new ArrayList<>(List.of(1, 2, 3, 4, 5));  
  
for (int i = 0; i < numbers.size(); i++) {  
    numbers.set(i, numbers.get(i) * 2);  
}  
  
int sum = 0;  
for (int i = 0; i < numbers.size(); i++) {  
    sum += numbers.get(i);  
}  
  
System.out.println(sum);
```


변수를 클래스로 옮기기

다음 변수 a 를 Something 이라는 클래스로 옮기려고 한다.

리팩토링 정의에 따르자면 겉보기 동작에 변화가 없는 채로 코드 재구성이 이뤄져야 하는데, 이때 어떤 순서로 작업을 해야 안전하게 옮길 수 있을까?

```
int a = 10; // 선언  
a += 5;    // 변경  
System.out.print(a); // 읽기
```

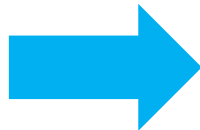


```
Something s = new Something(10);  
s.addA(5);  
System.out.print(s.getA());
```

[리팩토링 기법] 필드 캡슐화하기

public 필드의 캡슐화가 필요한 경우, 필드를 private 으로 만들고 접근자를 제공할 수 있다.

```
public String name;
```



```
private String _name;  
public String getName(){  
    return _name;  
}  
public void setName(String arg) {  
    _name = arg;  
}
```

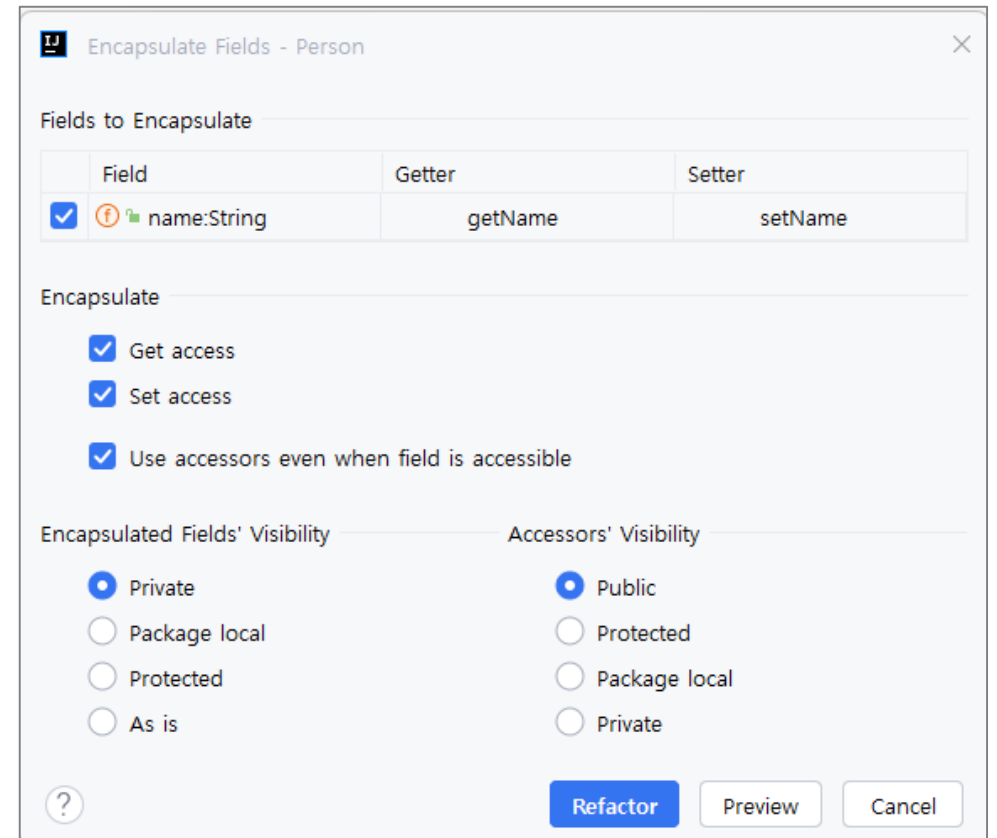
refactor this 로 필드 캡슐화하기

getter 와 setter 를 이용하도록 변경한다.

```
class Person {  
    public String name;  
}
```

refactor this -> encapsulate field

```
class Service {  
    public void doSomething(Person p){  
        p.name += " 고객님";  
        System.out.println(p.name);  
    }  
}
```



리팩토링 후 테스트에서 위임 사용하기

테스트 코드는 리팩토링 이전 기준으로 작성되어 있기 때문에,
리팩토링한 코드가 올바르게 동작하는지 확인하기 위해서는 우선 위임(delegation)을 이용한다.

